

Chapter 6

Implementation of the AAST Connect Platform

Ready-to-use graduation project chapter draft based on the final-app-combined branch for students and fresh graduates, and the adminFinal branch for the staff/admin portal.

Document note: The two pasted chapter files supplied with the request contained the same older implementation text. This version keeps the useful structure but updates the content to match the actual final application branches, including student, fresh graduate, admin, real-time notification, document, and AI chatbot implementation details.

6.1 Introduction

This chapter describes the technical implementation of the AAST Connect platform. The implementation translated the requirements, use cases, and design artifacts from the previous chapters into a working Flutter-based system connected to a Supabase backend. The platform is implemented as two closely related application surfaces: a student and fresh graduate mobile application, and an administrative staff portal. Both surfaces communicate with the same backend data model for users, vacancies, applications, documents, training records, notifications, and approval decisions.

The student side focuses on training-hour submission, opportunity discovery, application tracking, profile management, document uploads, and chatbot support. The fresh graduate side focuses on job and internship opportunities, graduate profile readiness, document management, saved/application states, and status tracking. The administrative side supports opportunity management, dashboard monitoring, student profile review, application approval or rejection, and training-hour verification.

6.2 Development Environment

AAST Connect was developed using a cross-platform mobile stack with Flutter and Dart, supported by Supabase for backend services and Google Gemini for the in-app assistant. The implementation uses a modular Flutter project structure where screens, widgets, services, models, providers, and role-specific folders are separated to reduce coupling between the student, fresh graduate, and admin experiences.

Component	Technology / Implementation
Programming language	Dart
Application framework	Flutter
Student/fresh-grad state management	Provider and ChangeNotifier
Admin state management	Flutter Riverpod for admin data flows, Provider for theme state

Component	Technology / Implementation
Backend	Supabase: PostgreSQL database, RPC functions, storage, realtime channels, and generated API access
AI chatbot	Google Gemini 2.5 Flash through the google_generative_ai package
Configuration	flutter_dotenv and .env loading in the combined student/fresh-grad app
Persistence	shared_preferences for local read-state and lightweight client persistence
File handling	file_picker and image_picker for CVs, certificates, photos, and supporting documents
External links	url_launcher for externally hosted application flows
Responsive UI	flutter_screenutil and responsive_builder
UI packages	lucide_icons, timelines_plus, flutter_slidable, awesome_snackbar_content
Testing	Flutter unit/widget tests, manual QA, Supabase connection checks, and workflow testing
Version control	Git and GitHub branches: final-app-combined and adminFinal

6.3 System Architecture Realization

The final implementation follows a layered client architecture. Flutter widgets are responsible for rendering the interface, providers hold local UI state, service classes encapsulate backend operations, and models convert database rows into typed application objects. Supabase acts as the shared backend layer for structured records, file storage, realtime notification events, and stored procedures used by authentication and admin access.

Layer	Implementation role
Presentation layer	Flutter screens and reusable widgets for dashboards, profiles, opportunities, notifications, support, approvals, and admin navigation.
State layer	Provider/ChangeNotifier in the combined app; Riverpod StateNotifier providers in the admin portal.
Service / datasource layer	Supabase service classes for vacancies, identity resolution, applications, training records, documents, notifications, fresh graduate home data, and admin datasources.
Domain layer	Admin branch separates entities, repositories, and use cases for dashboard, opportunities, approvals, and student profiles.
Data layer	Supabase tables and views/functions including users, student, freshgraduate, profile, vacancies, application, trainingrecord, document, notification, student_application_counts, vacancy_applicant_counts, admin_fetch_students, admin_fetch_trainingrecords, signin_admin, verify_login, and set_session_context.
External services	Gemini API for AI assistance, Supabase Storage for uploaded files, and external URLs for opportunities using external application methods.

6.3.1 Role Separation

The system separates behavior by role at sign-in time. The student/fresh-graduate app resolves the submitted college ID and password through the `verify_login` Supabase RPC, stores the active session in a `UserSession` singleton, and routes users to either `StudentMainNavigation` or `FreshGradNavigation`. Admin users are intentionally not routed inside the combined mobile app; they use the separate staff portal implemented in the `adminFinal` branch. The admin portal authenticates using the `signin_admin` RPC and stores staff context in `AuthSession`.

6.3.2 Backend Communication Pattern

All major workflows communicate with Supabase through service or datasource classes. The combined app uses `Supabase.instance.client` directly inside service classes such as `VacancyService`, `FreshGradVacancyService`, `NotificationService`, `FreshGradHomeService`, and `StudentIdentityService`. The admin portal uses datasource classes such as `DashboardRemoteDataSource`, `OpportunityRemoteDataSource`, `StudentRemoteDataSource`, and `ApprovalRemoteDataSource`, then exposes the behavior through repositories, use cases, and Riverpod providers.

6.4 Authentication and Session Management

Authentication is based on university or college ID rather than third-party OAuth. In the student/fresh-graduate application, the login form validates the registration number and password, calls `verify_login`, and resolves the returned user role. The resulting session stores the user ID, student ID when applicable, role, name, and college ID. This prevents screens from relying on hard-coded user IDs and allows profile, applications, documents, notifications, and chatbot context to load for the currently signed-in user.

The admin portal has a separate login flow through `signin_admin`. After successful login, the portal stores admin identity values in `AuthSession` and refreshes database session context using `set_session_context`. This pattern allows admin-only RPC functions and database policies to understand which staff user is active before fetching protected student, application, and training data.

- Student users are routed to `StudentMainNavigation`.
- Fresh graduate users are routed to `FreshGradNavigation` and their freshgraduate profile is loaded.
- Admin users are handled by the separate AAST Connect Staff portal.
- Logout and role switches clear cached providers, chat session data, and the local `UserSession` state.

6.5 Student Application Implementation

The student implementation is organized under student screens, shared services, and profile widgets. The main student navigation provides access to home, opportunities, tracking, support, notifications, and profile pages. The design uses reusable components such as custom navigation bars, status chips, document cards, profile modals, progress cards, deadline cards, opportunity cards, and modal form fields to keep the user interface consistent.

6.5.1 Student Home and Training Progress

The student home screen reads profile and training data from the student and profile tables, loads recent vacancies from the vacancies table, and presents upcoming deadlines. Training progress is calculated through TrainingService using the canonical student.completedtraininghours and student.requiredtraininghours fields. The result includes completed hours, required hours, remaining hours, extra hours, completion state, and progress percentage.

6.5.2 Opportunity Discovery and Application

Student opportunities are fetched from vacancies where the target audience is STUDENT or BOTH. The opportunity experience supports browsing, search and filtering, details modals, saved programs, and application submission. Internal applications insert a row into the application table with PENDING status, applicant identity, college ID, cover letter, submission date, and an optional document snapshot. External opportunities use the stored application method and URL to redirect users outside the app when the employer requires an external application process.

A key implementation detail is document snapshotting. When a user applies with an uploaded document, the service downloads the original file from the documents storage bucket, uploads a detached copy under an applications path, creates a separate document row for that snapshot, and stores the snapshot document ID on the application. This keeps the submitted application evidence stable even if the user later edits or deletes the original profile document.

6.5.3 Profile, Documents, and Training-Hour Submission

The student profile screen loads student, profile, document, and saved-program data. Profile widgets allow editing student information, skills, interests, portfolio information, documents, and training-hour submissions. File uploads use Supabase Storage and create document table rows with document type and public file path. The training-hours modal allows a student to submit organization and training details, select or upload evidence, and insert a trainingrecord row for admin review.

6.5.4 Tracking and Cancellation

The student tracking screen loads application rows and trainingrecord rows for the current user. Application and training statuses are displayed with reusable status chips and timeline-style UI patterns. Users can cancel pending applications or training records, which updates the status to CANCELED rather than deleting the historical record. This preserves auditability while giving users control over active submissions.

6.6 Fresh Graduate Application Implementation

The fresh graduate implementation is organized under the fresh_grads folder with separate screens, widgets, models, theme definitions, and utility providers. It reuses the shared Supabase backend but adjusts the workflow toward job readiness, employment opportunities, graduate documents, and application tracking rather than mandatory training-hour completion.

6.6.1 Fresh Graduate Dashboard

FreshGradHomeService builds the fresh graduate dashboard by loading recent application rows, all graduate-targeted vacancies, and the latest opportunities. It calculates total applications, pending applications, approved applications, rejected applications, job matches, applied vacancy IDs, recent activities, and latest opportunities. The service also uses a two-minute cache per user to reduce repeated Supabase requests while keeping the dashboard responsive.

6.6.2 Graduate Opportunities and Applications

Fresh graduate opportunities are filtered from vacancies where `target_audience` is GRADUATE or BOTH. FreshGradVacancyService loads all opportunities, latest opportunities, and already-applied vacancy IDs. For internal applications, it resolves the current user, optionally snapshots the selected document, and inserts the application row with applicant name, college ID, cover letter, status, and timestamps. This mirrors the student application flow while using fresh graduate identity and profile data.

6.6.3 Graduate Profile and Document Library

The fresh graduate profile implementation loads the `freshgraduate` row by college ID and manages graduate-specific fields such as major, GPA, phone, bio, skills, interests, and portfolio links. The document section allows users to upload, list, and delete files from the documents bucket and document table. Deletion is guarded by application links: documents already attached to applications are not removed in a way that would break existing submitted applications.

6.6.4 Fresh Graduate Notifications

Fresh graduates receive database notifications through the shared notification service and also receive opportunity notification behavior for newly available vacancies. The fresh graduate opportunity notification service uses `shared_preferences` to track seen vacancy IDs locally and can create unread opportunity notification items from the latest vacancies targeted to graduates.

6.7 Admin Portal Implementation

The `adminFinal` branch implements a separate staff-facing portal named AAST Connect Staff. Unlike the combined student/fresh-graduate app, the admin portal uses a cleaner layered structure with core dependency injection, data datasources, data models, repository implementations, domain entities, domain repositories, use cases, presentation viewmodels, screens, widgets, and theme files. This structure makes approval, dashboard, opportunity, and student profile workflows easier to maintain independently from the mobile user-facing app.

6.7.1 Admin Dashboard

The dashboard module loads summary data from `student_application_counts`, recent application rows, and recent `trainingrecord` rows. It calculates total applications, pending applications, training uploads, and a combined recent activity feed. Training activity rows are enriched with student names from the student table so staff can quickly understand who submitted each item.

6.7.2 Opportunity Management

The opportunity management module provides paginated access to active vacancies, search by title or company name, filtering by type, applicant counts through `vacancy_applicant_counts`, and create, update, and delete operations on the vacancies table. The admin form supports fields such as title, company, location, description, required skills, company logo URL, work mode, paid status, deadline, target audience, application method, and external application URL. When `application_method` is `EXTERNAL`, the external URL is required before saving.

6.7.3 Application Approval Workflow

The approval module fetches applications from the application table and enriches them with vacancy company data, student or fresh graduate GPA, profile image, and applicant information. Admins can approve applications by updating status to `APPROVED`, or reject them by setting status to `REJECTED` with an optional rejection reason. These status and reason fields are then visible to students and fresh graduates through their tracking screens and chatbot context.

6.7.4 Training-Hour Verification Workflow

Training verification uses the `admin_fetch_trainingrecords` RPC to fetch pending or historical training records. Rows are enriched with student name, college ID, profile image, and completed training hours. When an admin approves a training record, the system reads the submitted hours, adds them to `student.completedtraininghours`, and marks the trainingrecord as `APPROVED`. When a record is rejected, the system stores status `REJECTED` and an optional rejection reason without increasing completed hours.

6.7.5 Student Profile Monitoring

The student profile section uses `admin_fetch_students` and `admin_fetch_application_counts` to display paginated student profile cards with application count, completed training hours, required training hours, and remaining hours. This gives administrators a single view of student progress and application activity.

6.8 Database, Storage, and Realtime Implementation

Supabase provides the main backend infrastructure. The application stores structured data in PostgreSQL tables and uses Supabase Storage for uploaded documents and certificates. The most important tables used by the final branches include `users`, `student`, `freshgraduate`, `profile`, `vacancies`, `application`, `trainingrecord`, `document`, `notification`, and `saved_programs`. The admin branch also uses database views or RPC functions such as `student_application_counts`, `vacancy_applicant_counts`, `admin_fetch_students`, `admin_fetch_application_counts`, `admin_fetch_trainingrecords`, `signin_admin`, `verify_login`, and `set_session_context`.

Backend object	Purpose in implementation
<code>users</code>	Stores general user identity, name, role, and college ID used by login and chat context.
<code>student</code>	Stores student profile information and training-hour counters.
<code>freshgraduate</code>	Stores fresh graduate profile data such as major, GPA, phone, bio, skills, interests, and portfolio links.
<code>profile</code>	Connects users to uploaded documents and profile-level data.
<code>vacancies</code>	Stores opportunities, target audiences, deadlines, application methods, and external URLs.

Backend object	Purpose in implementation
application	Stores user submissions, status, rejection reason, cover letter, timestamps, and attached document snapshot ID.
trainingrecord	Stores student training-hour submissions, evidence, status, and approval/rejection reason.
document	Stores document metadata and Supabase Storage file URLs.
notification	Stores user notifications, read state, message text, and creation timestamps.
saved_programs	Stores saved student opportunities.

Realtime updates are used for notification and state freshness. The student and fresh graduate navigation components subscribe to Supabase realtime channels for notification inserts and key table changes such as applications, training records, students, and vacancies. When a relevant change arrives, the UI can refresh unread counts, invalidate cached chat context, and show an in-app snackbar notification.

6.9 AI Chatbot Implementation

AAST Connect includes an AI support assistant powered by Gemini 2.5 Flash through the `google_generative_ai` package. The `ChatService` creates a `GenerativeModel` with a system instruction that restricts the assistant to user-facing student and fresh graduate support. The prompt specifically prevents the assistant from discussing admin features or database internals and guides it to use different behavior for students and fresh graduates.

The chatbot is connected to live user context through `CachedChatProvider`. Before sending a user question to Gemini, the provider builds a hidden context block from Supabase. For students, it includes training-hour requirements, completed hours, recent applications, and latest student opportunities. For fresh graduates, it includes profile information, recent graduate applications, rejection reasons when available, and latest graduate opportunities. The context is cached for five minutes per role/user session and invalidated when app data changes.

- Student assistant focus: training hours, applications, documents, and student opportunities.
- Fresh graduate assistant focus: job applications, profile readiness, documents, and graduate opportunities.
- Session separation prevents chat history and cached context from leaking between different user roles.
- Network or API failures return a friendly fallback message rather than crashing the support screen.

6.10 Notifications and User Feedback

The final implementation uses in-app and database-backed notifications rather than leaving the notification mechanism as a placeholder. `NotificationService` reads notification rows for a user, counts unread items, marks one or all items as read, deletes notification rows where appropriate, formats timestamps, extracts rejection reasons, and maps messages to suitable icons and colors. Realtime subscriptions listen for inserted notification rows filtered by user ID so users can be alerted while using the app.

User feedback is also provided through validation messages, loading indicators, status badges, snackbars, dialogs, and empty states. The student/fresh-graduate app uses professional app chrome, reusable status chips, cancellation confirmation dialogs, and modal forms. The admin portal uses skeleton loading components, approval/rejection dialogs, search and filter controls, and confirmation dialogs for destructive

actions such as deleting opportunities.

6.11 Security and Data Integrity

Several implementation choices improve security and data integrity. Sign-in is resolved through database RPC functions instead of client-side role guessing. The active user session is stored locally in memory and cleared during logout. The admin portal sets database session context before protected admin operations. Uploaded documents are stored in Supabase Storage and referenced through database rows rather than being embedded in application state.

- Role-based routing separates student, fresh graduate, and admin user journeys.
- Document snapshotting preserves the exact file attached to an application submission.
- Application and training cancellation use status updates rather than hard deletion.
- Training approval updates both the trainingrecord status and the student's completedtraininghours counter.
- Rejection reasons are stored with rejected applications or training records for transparent feedback.
- Configuration values for the combined app are loaded from .env through flutter_dotenv.

6.12 Testing and Verification

Testing combined automated checks with manual workflow verification. The combined student/fresh-graduate branch contains focused Flutter tests for identity resolution and chat session behavior. The identity tests verify that student and fresh graduate sessions are resolved correctly from database-style rows. The chat session tests verify that messages and cached contexts remain separated between student and fresh graduate sessions and can be invalidated without affecting other sessions.

Manual testing was also necessary because many workflows depend on Supabase data, uploaded files, realtime updates, and role-specific navigation. The team validated login behavior, opportunity browsing, application submission, document upload, training-hour submission, notification read states, admin approval/rejection actions, and status updates across student, fresh graduate, and staff interfaces.

Test / verification area	Purpose
student_identity_service_test	Validates role and session resolution for students and fresh graduates.
chat_session_store_test	Validates per-session message/context caching and invalidation.
chat_session_scope_test	Validates session-key changes across role/user combinations.
widget_test	Provides baseline Flutter widget-test structure.
Manual Supabase workflow testing	Validates login, profile loading, application insertion, training approval, document storage, and notification flows against the live backend.

6.13 Challenges Encountered

The implementation phase involved several challenges because AAST Connect combines multiple user roles, document-heavy workflows, realtime updates, and AI support in one platform.

- University ID authentication required custom RPC-based login and careful mapping between users, student records, and fresh graduate records.
- The team had to separate the admin portal from the student/fresh-graduate app so staff-only functionality would not appear in the user-facing mobile app.
- Document handling required special care because applications must keep evidence stable even when a user edits or removes a profile document later.
- Training-hour approval had to update both the record status and the student's accumulated completed hours.
- The chatbot needed strict prompt boundaries and live context caching so it could personalize answers without exposing database details or admin functionality.
- Realtime notifications and unread counts required coordination between Supabase subscriptions, notification table state, and local UI refresh behavior.
- The final implementation had to support both internal applications and external application URLs within one opportunity workflow.

6.14 Implementation Outcomes

The final implementation produced a working platform that supports the core objectives of AAST Connect. Students can sign in, view suitable opportunities, manage documents, apply to internal opportunities, track application status, submit training-hour evidence, monitor training progress, receive notifications, and ask the AI assistant for help. Fresh graduates can sign in, maintain a job-readiness profile, upload documents, browse graduate opportunities, submit applications, track decisions, receive opportunity notifications, and use the assistant for graduate-specific support.

Administrators can sign in to a dedicated staff portal, view dashboard statistics, manage opportunities, monitor student profiles, review applications, approve or reject submissions, verify training-hour records, upload or view supporting documents, and record rejection reasons. The shared Supabase backend keeps the three role experiences connected while the Flutter codebase keeps role-specific screens and services separated.

Overall, the implementation demonstrates a complete end-to-end workflow from opportunity creation to user application, admin review, user notification, training progress update, and AI-assisted support. This satisfies the project's goal of improving opportunity discovery, application tracking, and training-hours verification for AAST students and fresh graduates.

Appendix A: Suggested Continuation If Appending After an Admin-Only Section

If the current graduation document already contains a detailed admin implementation section and you only want to continue after it, the following paragraphs can be inserted immediately after that admin section. They focus on the student and fresh graduate implementation that the older draft did not cover fully.

Student and Fresh Graduate Mobile Implementation Continuation

In addition to the staff portal, AAST Connect includes a combined mobile application for students and fresh graduates. This application uses Flutter, Provider, Supabase, flutter_dotenv, file_picker, image_picker, shared_preferences, flutter_screenutil, and google_generative_ai. At login, the application verifies the submitted college ID and password through the verify_login Supabase RPC, stores the active user's role and identifiers in UserSession, and routes the user to the correct interface. Students are routed to StudentMainNavigation, while fresh graduates are routed to FreshGradNavigation. Admin users are not opened in the combined app because they use the dedicated staff portal.

The student implementation provides a complete training and opportunity workflow. Students can view dashboard cards, track completed and required training hours, browse vacancies targeted to STUDENT or BOTH, save programs, upload CVs and certificates, submit internal applications, cancel active submissions, and monitor application or training statuses. Training-hour progress is calculated from the student table using completedtraininghours and requiredtraininghours, while training submissions are stored in the trainingrecord table for admin approval.

The fresh graduate implementation reuses the same backend but adapts the experience for employment readiness. Fresh graduates can edit profile fields such as major, GPA, bio, phone, skills, interests, and portfolio links; upload and manage documents; browse vacancies targeted to GRADUATE or BOTH; apply to internal opportunities; and view dashboard statistics such as total, pending, approved, and rejected applications. FreshGradHomeService caches dashboard data for a short period to improve responsiveness while still allowing refreshes after major data changes.

Both user-facing roles share the AI support assistant. Before a message is sent to Gemini 2.5 Flash, the app builds a hidden live-context summary from Supabase. Student context includes training hours, recent applications, and latest student opportunities. Fresh graduate context includes graduate profile fields, recent job applications, rejection reasons, and latest graduate opportunities. The assistant is instructed to remain within the AAST Connect support domain and not discuss admin features or database internals.

The final mobile application also implements database-backed notifications. NotificationService fetches notification rows, counts unread items, marks notifications as read, formats timestamps, maps notification messages to icons and colors, and listens to Supabase realtime inserts for the active user. This replaced the earlier placeholder notification description with an implemented in-app notification mechanism.